

Instituto por Cooperativa Villa Hermosa

Curso: Programación II

Fase 3

Desarrollo / Código

Nombre: Yuli Gabriela Villagrán Ramírez

Grado: 5to. PC

Sección: A

Clave: 33

la_fortaleza_de_los_glitches.py > ...

```
1  """
2  Yuli Gabriela Villagrán Ramírez - 5to. PC "A"
3  """
4
5  import os
6  import random
7
8  # =====
9  # ETAPA 3: DESARROLLO DEL CÓDIGO
10 # =====
11
12 """
13 =====
14 CLASE PADRE (SUPERCLASE): ElementoJuego
15 -----
16 Representa cualquier entidad u objeto base que existe dentro del videojuego.
17 Define los atributos comunes (nombre, emoji o símbolo y coordenadas X, Y)
18 y el método base 'mostrar_info()' que será sobrescrito por sus clases hijas
19 para aplicar POLIMORFISMO.
20 =====
21 """
22 class ElementoJuego:
23     def __init__(self, nombre, simbolo, x=0, y=0):
24         self.nombre = nombre
25         self.simbolo = simbolo
26         self.x = x
27         self.y = y
28
29     # Método base para el polimorfismo
30     def mostrar_info(self):
31         return f"Entidad: {self.nombre} | Símbolo: {self.simbolo} | Posición: ({self.x}, {self.y})"
32
```

```
33
34 """
35 =====
36 CLASE HIJA 1 (SUBCLASE): Personaje (Hereda de ElementoJuego)
37 -----
38 Representa al héroe controlado por el jugador.
39 Hereda el nombre, símbolo (👤) y posición, y agrega atributos como
40 salud (corazones), energía, monedas e inventario (mochila).
41 =====
42 """
43 class Personaje(ElementoJuego):
44     def __init__(self, nombre):
45         super().__init__(nombre, "👤", x=0, y=0)
46         self.salud = 3          # 3 corazones de vida
47         self.energia = 25      # 25 puntos de energía para el Nivel 1
48         self.monedas = 0       # Contador de monedas
49         self.mochila = []       # Inventario de herramientas
50
```

```

50
51 # OCURRE EL POLIMORFISMO: Se rediseña mostrar_info() para mostrar el HUD del jugador
52 def mostrar_info(self):
53     corazones = "❤️ " * max(0, self.salud)
54     return f"Jugador: {self.nombre} {self.simbolo} | Posición: ({self.x},{self.y}) |
    Vidas: {corazones}({self.salud}) | Energía: {self.energia} | Monedas: {self.monedas}
    🏠"
55
56 def reiniciar_estadisticas(self):
57     """Restablece la salud, energía y posición para iniciar una nueva partida"""
58     self.salud = 3
59     self.energia = 25
60     self.x = 0
61     self.y = 0
62
63
64 """
65 =====
66 CLASE HIJA 2 (SUBCLASE): Enemigo (Hereda de ElementoJuego)
67 -----
68 Representa las amenazas del juego como los Glitches o el Rey Glitch.
69 Hereda la estructura base de entidad e incorpora el nivel de daño.
70 =====
71 """
72 class Enemigo(ElementoJuego):
73     def __init__(self, nombre, dano=1, x=0, y=0):
74         super().__init__(nombre, "💀", x, y)
75         self.dano = dano
76
77 # OCURRE EL POLIMORFISMO: Muestra los datos de la amenaza
78 def mostrar_info(self):
79     return f"¡AMENAZA! {self.nombre} {self.simbolo} | Daño: {self.dano} corazón(es)"
80

```

```

81
82 """
83 =====
84 CLASE HIJA 3 (SUBCLASE): ObjetoInteractable (Hereda de ElementoJuego)
85 -----
86 Representa los elementos recolectables o destructibles (madera, rocas, objetos).
87 =====
88 """
89 class ObjetoInteractable(ElementoJuego):
90     def __init__(self, nombre, simbolo, funcion, x=0, y=0):
91         super().__init__(nombre, simbolo, x, y)
92         self.funcion = funcion
93
94 # OCURRE EL POLIMORFISMO: Describe la utilidad o función del objeto
95 def mostrar_info(self):
96     return f"Objeto: {self.nombre} {self.simbolo} - Descripción: {self.funcion}"
97
98

```

```

99 # =====
100 # FUNCIONES AUXILIARES Y MANEJO DE PANTALLA
101 # =====
102
103 def limpiar_pantalla():
104     """Limpia la terminal según el sistema operativo (Windows/Linux/Mac)."""
105     os.system('cls' if os.name == 'nt' else 'clear')
106
107 def pausar():
108     """Pausa la ejecución hasta que el usuario presione Enter."""
109     input("\nPresiona [ENTER] para continuar...")
110
111 def dibujar_matriz_6x6(jugador):
112     """Renderiza visualmente el mapa de la cuadrícula 6x6 con la posición del jugador."""
113     print("\n--- MAPA DEL BOSQUE (6x6) ---")
114     for y in range(6):
115         fila = ""
116         for x in range(6):
117             if x == jugador.x and y == jugador.y:
118                 fila += f"[{jugador.simbolo}]"
119             else:
120                 fila += "[■]"
121         print(fila)
122     print("-----\n")
123
124 # =====
125 # ETAPA 2: FLUJO DE CADA OPCIÓN DEL MENÚ (LÓGICA Y NIVELES)
126 # =====
127
128
129 def nivel_1_bosque(jugador):
130     """
131     NIVEL 1: El Bosque de Píxeles (Exploración y Recursos)
132     - Matriz 6x6
133     - Energía inicial = 25 (resta 1 por paso)
134     - Objetivo: Juntar 3 maderas (🌲)
135     - Regla especial: 5 monedas restauran la vida al máximo (3 ❤️)
136     """
137     limpiar_pantalla()
138     print("=====")
139     print("  NIVEL 1: EL BOSQUE DE PÍXELES (Mundo 1 - Matriz 6x6)  ")
140     print("=====")
141     print("Objetivo: Recolecta 3 maderas (🌲) antes de quedarte sin energía.\n")
142

```

```

143     maderas_recolectadas = 0
144     jugador.energia = 25 # Reiniciar energía para el nivel
145     jugador.x, jugador.y = 0, 0 # Posición inicial en la esquina superior izquierda
146
147     # Instanciamos herramientas y enemigos con Polimorfismo
148     pico = ObjetoInteractuable("Pico de Datos", "🔪", "Destruye rocas (🪨) que bloquean el camino")
149     glitch = Enemigo("Glitch de Código", dano=1)
150

```

```

151 # Ciclo principal del Nivel 1
152 while maderas_recolectadas < 3 and jugador.energia > 0 and jugador.salud > 0:
153     limpiar_pantalla()
154     print("=====")
155     print("                     NIVEL 1: EN CURSO                     ")
156     print("=====")
157     # Demostración del polimorfismo imprimiendo el HUD
158     print(jugador.mostrar_info())
159     print(f"Progreso Maderas: {maderas_recolectadas}/3 📊")
160
161     # Renderizar la matriz 6x6 en pantalla
162     dibujar_matriz_6x6(jugador)
163
164     print("Controles: [W] Arriba | [S] Abajo | [A] Izquierda | [D] Derecha")
165     print("                [E] Abrir Mochila      | [Q] Pedir Pista (2 📊)")
166     print("-----")
167
168     # Manejo de errores try-except para los controles de usuario
169     try:
170         paso = input("Ingresa tu movimiento: ").strip().lower() # Tolerancia a mayúsculas
171         if paso not in ['w', 'a', 's', 'd', 'e', 'q']:
172             raise ValueError("Tecla invalida")
173     except ValueError:
174         print("⚠ Error: Usa únicamente las teclas W, A, S, D, E o Q.")
175         pausar()
176         continue
177
178     # Uso de Mochila / Herramientas (Tecla E)
179     if paso == 'e':
180         print(f"\n🎒 MOCHILA:")
181         print(pico.mostrar_info())
182         pausar()
183         continue
184

```

```

185 # Pista (Tecla Q)
186 if paso == 'q':
187     if jugador.monedas >= 2:
188         jugador.monedas -= 2
189         print("\n💡 PISTA: Muévete hacia los bordes para encontrar maderas ocultas.")
190     else:
191         print("\n⚠ No tienes suficientes monedas para solicitar una pista (Cuesta 2 📊).")
192     pausar()
193     continue
194
195 # Lógica de movimiento en la Matriz 6x6
196 if paso == 'w' and jugador.y > 0:
197     jugador.y -= 1
198 elif paso == 's' and jugador.y < 5:
199     jugador.y += 1
200 elif paso == 'a' and jugador.x > 0:
201     jugador.x -= 1
202 elif paso == 'd' and jugador.x < 5:
203     jugador.x += 1
204 else:
205     print("\n🛑 ¡Cuidado! Chocaste con el borde de la matriz 6x6.")
206

```

```

207     # Restar 1 de energía por paso efectuado
208     jugador.energia -= 1
209
210     # Eventos aleatorios del escenario
211     evento = random.randint(1, 10)
212
213     if evento in [1, 2, 3]:
214         maderas_recolectadas += 1
215         print("\n🪵 ¡Encontraste y recolectaste 1 bloque de madera!")
216     elif evento in [4, 5]:
217         jugador.monedas += 5
218         print("\n💰 ¡Encontraste un cofre con 5 Monedas!")
219         # Requerimiento: Juntar 5 monedas restaura la salud a 3 corazones
220         if jugador.salud < 3:
221             jugador.salud = 3
222             print("❤️ ¡Tus 5 monedas han restaurado tu salud al máximo (3 ❤️)!")
223     elif evento == 6:
224         print(f"\n{glitch.mostrar_info()}")
225         if "Escudo Protector" in jugador.mochila:
226             print("🛡️ ¡Tu Escudo Protector absorbió el ataque del Glitch y se rompió!")
227             jugador.mochila.remove("Escudo Protector")
228         else:
229             jugador.salud -= 1
230             print("💀 ¡El Glitch te atacó! Perdiste 1 corazón de vida ❤️.")
231
232     pausar()
233
234     # Evaluación de fin de nivel
235     if jugador.salud <= 0 or jugador.energia <= 0:
236         print("\n❌ Nivel Fallido: Te has quedado sin energía o vidas.")
237         return False
238
239     print("\n🏆 ¡NIVEL 1 COMPLETADO! Maderas recolectadas. ¡Mapa Mundo 2 Desbloqueado!")
240     pausar()
241     return True
242
243

```

```

244 def nivel_2_carrera(jugador):
245     """
246     NIVEL 2: La Carrera de los Números (Obstáculos y Acertijo)
247     - Obstáculo en la pista
248     - Reto matemático / secuencia lógica
249     """
250     limpiar_pantalla()
251     print("=====")
252     print("  NIVEL 2: LA CARRERA DE LOS NÚMEROS (Mundo 2)  ")
253     print("=====")
254     print("Avanzando por la pista lineal... 🏃➡️")
255     print("🚧 ¡PELIGRO! Un obstáculo bloquea el camino.")
256     print("Para saltar la tubería, resuelve la secuencia lógica:")
257     print("\n ->  3, 6, 12, ¿...?  <-  \n")
258

```

```

259 # Manejo de errores try-except para validar la respuesta numérica
260 try:
261     respuesta = int(input("Introduce el número correcto para saltar: ").strip())
262     if respuesta == 24:
263         print("\n✅ ¡CORRECTO! El patrón se duplica (12 x 2 = 24).")
264         print("🏆 ¡Saltaste el obstáculo con éxito y cruzaste la meta!")
265         pausar()
266         return True
267     else:
268         print("\n❌ Respuesta incorrecta. Chocaste contra la tubería y perdiste el nivel.")
269         jugador.salud -= 1
270         pausar()
271         return False
272 except ValueError:
273     print("\n⚠️ Error: Debes ingresar un valor numérico entero.")
274     jugador.salud -= 1
275     pausar()
276     return False
277
278
279 def nivel_3_fortaleza(jugador):
280     """
281     NIVEL 3: El Núcleo de la Fortaleza (Examen Final de Lógica)
282     - 3 Preguntas lógicas consecutivas
283     - Si acierta las 3: Despliega el trofeo ASCII de victoria
284     """
285     limpiar_pantalla()
286     print("=====")
287     print(" NIVEL 3: EL NÚCLEO DE LA FORTALEZA (Mundo 3 - Jefe Final)")
288     print("=====")
289     print("👹 El Rey Glitch bloquea la salida con 3 candados lógicos.")
290     print("Debes responder correctamente las 3 preguntas consecutivas.\n")
291
292     preguntas = [
293         ("1. ¿Qué estructura de control se utiliza para repetir un bloque de código mientras una condición sea verdadera?\n 1) if-else\n 2) while\n 3) try-except", 2),
294         ("2. ¿Qué bloque de código se utiliza en Python para capturar y manejar errores previniendo cierres inesperados?\n 1) try-except\n 2) for-in\n 3) def", 1),
295         ("3. ¿Cuál es la palabra clave para definir una función en Python?\n 1) class\n 2) function\n 3) def", 3)
296     ]
297

```

```

298 aciertos = 0
299
300 # Ciclo for para evaluar las preguntas
301 for p_texto, r_correcta in preguntas:
302     print("-----")
303     print(p_texto)
304     try:
305         opcion = int(input("\nTu respuesta (1-3): ").strip())
306         if opcion == r_correcta:
307             print("✅ ¡Candado desbloqueado!")
308             aciertos += 1
309         else:
310             print("❌ Respuesta incorrecta. El candado se atascó.")
311     except ValueError:
312         print("⚠️ Entrada no válida. Contada como respuesta incorrecta.")
313

```

```

314 # Verificación final de victoria
315 if aciertos == 3:
316     limpiar_pantalla()
317     print("=====")
318     print("          🏆 ¡FELICITACIONES! ¡HAS GANADO! 🏆          ")
319     print("=====")
320     print(f"    ¡{jugador.nombre}, ERES EL MAESTRO SUPREMO DE LA LÓGICA! ")
321     print("""
322
323         '._==_._==_._.'
324         .-\\:      /-.'
325         | (|:.     |) |
326         '-|:..    |-.'
327         \\:..     /
328         '._:.._.'
329         | ) (
330         _.' _.'
331         _-_-_-_-_-
332     """)
333     print("=====")
334     pausar()
335     return True
336 else:
337     print(f"\n❌ Has fallado. Acertaste {aciertos}/3 preguntas. La Fortaleza sigue cerrada.")
338     pausar()
339     return False
340
341
342 def flujo_jugar(jugador):
343     """Gestión secuencial de los 3 niveles del Diagrama de Flujo"""
344     jugador.reiniciar_estadisticas() # Garantizar que inicia con 3 vidas y 25 de energía
345     # Nivel 1
346     if nivel_1_bosque(jugador):
347         # Nivel 2
348         if nivel_2_carrera(jugador):
349             # Nivel 3
350             nivel_3_fortaleza(jugador)
351
352

```

```

353 def ver_inventario(jugador):
354     """Opción: Inventario y Vestuario (Herramientas)"""
355     limpiar_pantalla()
356     print("=====")
357     print("          INVENTARIO Y VESTUARIO (MOCHILA)          ")
358     print("=====")
359     print("1. Equipar Traje 1 (Pico de Datos) 🔧")
360     print("2. Equipar Traje 2 (Escudo Protector) 🛡️")
361     print("3. Equipar Traje 3 (Pociones de Energía) 💧")
362     print("4. Volver al Menú Principal")
363

```

```

364     try:
365         opc = int(input("\nSelecciona una opción (1-4): ").strip())
366         if opc == 1:
367             if "Pico de Datos" not in jugador.mochila:
368                 jugador.mochila.append("Pico de Datos")
369                 print("✅ ;Pico de Datos equipado en la mochila!")
370             else:
371                 print("📁 Ya tienes el Pico de Datos equipado.")
372         elif opc == 2:
373             if "Escudo Protector" not in jugador.mochila:
374                 jugador.mochila.append("Escudo Protector")
375                 print("✅ ;Escudo Protector equipado en la mochila!")
376             else:
377                 print("📁 Ya tienes el Escudo Protector equipado.")
378         elif opc == 3:
379             jugador.energia += 10
380             print("🍷 ;Poción consumida! Restauraste +10 puntos de energía.")
381         elif opc == 4:
382             return
383         else:
384             print("⚠️ Opción fuera de rango.")
385     except ValueError:
386         print("⚠️ Error: Ingresa un número entero.")
387
388     pausar()
389
390
391 def ver_opciones():
392     """Opción: Configuración del juego"""
393     limpiar_pantalla()
394     print("=====")
395     print("                                OPCIONES                                ")
396     print("=====")
397     print("🔊 Sonido: [Efectos: ACTIVADOS] | [Música: ACTIVADA]")
398     print("📧 Soporte: Correo de asistencia activo")
399     print("🔔 Notificaciones: Activadas")
400     print("📄 Créditos: Desarrollado para Programación II")
401     print("⚖️ Legal: Videojuego Educativo en Consola Python")
402     pausar()
403
404

```

```

405 def modo_multijugador():
406     """Opción: Modo 3 Jugadores"""
407     limpiar_pantalla()
408     print("=====")
409     print("                                MODO 3 JUGADORES                                ")
410     print("=====")
411     print("¿Primero en ganar 3 niveles?\n")
412

```

```

413     jugadores = []
414     for i in range(1, 4):
415         nombre = input(f"Ingrese el nombre del Jugador {i}: ").strip() or f"Jugador_{i}"
416         jugadores.append(Personaje(nombre))
417
418     print("\n🎮 Jugadores Registrados en la Sesión:")
419     for j in jugadores:
420         print(f" -> {j.mostrar_info()}") # Aplicación de Polimorfismo
421
422     pausar()
423
424
425 def ver_referencias():
426     """Opción: Tabla de Referencias"""
427     limpiar_pantalla()
428     print("=====")
429     print("          TABLA DE REFERENCIAS (TECLAS Y EMOJIS)          ")
430     print("=====")
431     print("👤 : Personaje principal (Vector)")
432     print("👾 : Enemigo Glitch / Rey Glitch")
433     print("🪵 : Madera (Recurso)")
434     print("🪨 : Roca (Obstáculo)")
435     print("💰 : Monedas (5 Monedas = Salud Máxima 3 ❤️)")
436     print("\nControles de Consola: [W, A, S, D] Movimiento | [E] Mochila")
437     pausar()
438
439
440 # =====
441 # ETAPA 1: BIENVENIDA Y MENÚ PRINCIPAL (ESTRUCTURA BASE)
442 # =====
443
444 def menu_principal():
445     """Bienvenida y Menú Principal con estructura a prueba de fallos"""
446     limpiar_pantalla()
447     print("=====")
448     print("      ¡BIENVENIDO A LA FORTALEZA DE LOS GLITCHES: PIXEL ADVENTURE!      ")
449     print("=====")
450     print("                      Cargando juego...                      ")
451     print("=====")
452
453     nombre_jugador = input("\n¿Cómo te llamas, explorador?: ").strip() or "Vector"
454     jugador = Personaje(nombre_jugador)
455

```

```

456 # Ciclo Principal (While True) del Menú
457 while True:
458     limpiar_pantalla()
459     print("=====")
460     print("                MENÚ PRINCIPAL                ")
461     print("=====")
462     print(f" Usuario activo: {jugador.nombre}")
463     print("-----")
464     print(" 1. Jugar")
465     print(" 2. Inventario / Mochila")
466     print(" 3. Opciones")
467     print(" 4. Modo 3 Jugadores")
468     print(" 5. Tabla de Referencias")
469     print(" 6. Salir")
470     print("=====")
471
472 # Captura y prevención de errores mediante try-except
473 try:
474     opcion = int(input("Selecciona una opción (1-6): ").strip())
475
476     if opcion == 1:
477         flujo_jugar(jugador)
478     elif opcion == 2:
479         ver_inventario(jugador)
480     elif opcion == 3:
481         ver_opciones()
482     elif opcion == 4:
483         modo_multijugador()
484     elif opcion == 5:
485         ver_referencias()
486     elif opcion == 6:
487         # Submenú confirmación de salida
488         confirmar = input("\n¿Está seguro de que desea salir? (s/n): ").strip().lower()
489         if confirmar == 's':
490             print("\n¡Gracias por jugar! ¡Hasta la próxima! 🍀")
491             break
492     else:
493         print("\n⚠️ Opción no válida. Ingresa un número entre 1 y 6.")
494         pausar()
495

```

```

496 except ValueError:
497     print("\n⚠️ Error: Ingresa únicamente un número entero.")
498     pausar()
499
500
501 # Punto de entrada para la ejecución del programa
502 if __name__ == "__main__":
503     menu_principal()

```